

df = pd.DataFrame(data.data, columns=data.feature_names)//df['house_prices'] = data.target

df.head() The head(20) method displays the first twenty rows of the DataFrame, allowing us to see what the data looks like. By default, head() shows the first five rows. **df.describe()** The describe() method provides summary statistics for each numeric column in the DataFrame, including count, mean, standard deviation, minimum, maximum, and quartile values. **df.isnull().sum()** The isnull() method creates a DataFrame of the same shape as our original data, but with Boolean values indicating whether each cell contains a missing value (True) or not (False). The sum() function then counts the number of True values in each column, giving us the total count of missing values.

Checking for missing values is essential because: Data completeness assessment, Informing preprocessing strategy, Data quality indicator, Bias prevention. **Seaborn scatterplot** sns.scatterplot(x='column_x', y='column_y', data=df) **pairplot** data_for_pairplots = df[['MedInc', 'AveBedrms', 'AveOccup', 'house_prices']]sns.pairplot(data=data_for_pairplots) **Correlation Heat Map** sns.heatmap(df.corr(), annot=True) Values range from -1 (perfect negative correlation) to 1 (perfect positive correlation), 0 indicates no linear correlation **Why Do We Split the Data?** Model Evaluation: The primary purpose of splitting data, Preventing Overfitting **train_test_split()** Parameters: X : The feature matrix (all columns except the target), y : The target variable (what we're trying to predict), test_size=0.2 : 20% of the data will be used for testing, 80% for training, random_state=42 Additional parameters: Stratification: train_test_split(X, y, stratify=y), Shuffle: train_test_split(X, y, shuffle=True) Data Leakage: Always split your data before performing any preprocessing, Scaling/normalization should be done after splitting to prevent information leakage, Small Test Sets: Too small a test set might not give reliable performance estimates, Too large a test set leaves less data for training, Non-Representative Splits: Ensure your split captures the full range of data variations, Consider using stratification for imbalanced datasets **Feature scaling** The StandardScaler standardizes features by removing the mean and scaling to unit variance. This means each feature will have a mean of 0 and a standard deviation of 1. fit_transform(): Computes the mean and std deviation and applies the scaling, transform(): Applies the scaling using the mean and standard deviation calculated from the training data. This prevents data leakage. **Common Pitfalls to Avoid** 1. Scaling Target Variables: Generally avoid scaling target variables in regression, Keep predictions in original units for interpretability 2. Not Considering Feature Distribution: StandardScaler assumes normal distribution, For skewed data, consider other scalers like RobustScaler or MinMaxScaler. **When Not to Scale** 1. Tree-based models: These algorithms are invariant to monotonic transformations, Scaling won't hurt but isn't necessary. 2. Binary features (0/1 values) No need to scale already normalized features. 3. Categorical variables: These should be encoded, not scaled **Linear Regression Model** model = LinearRegression() model.fit(X_train, y_train) y_pred = model.predict(X_test) **MSE** mse = mean_squared_error(y_test, y_pred) Formula: $MSE = (1/n) \sum (actual - predicted)^2$ We use squared error because: It penalizes larger errors more heavily, It treats positive and negative errors equally, It's differentiable, making it useful for optimization. **Hyperparameter Tuning** Ridge Regression (also known as L2 regularization) is a modification of Linear Regression that adds a penalty term to the model's loss function. This penalty is based on the square of the coefficients' magnitudes. By doing this, Ridge helps prevent overfitting by discouraging the model from using large coefficient values that might make it too sensitive to individual features. Higher alpha = stronger regularization = smaller coefficients. Lower alpha = weaker regularization = closer to standard Linear Regression param_grid = {'alpha': [1.0, 10, 100]}//grid_search = GridSearchCV(Ridge(), param_grid, cv=5)//grid_search.fit(X_train, y_train)// print(grid_search.best_params_) **joblib** joblib.dump(model, 'california_housing_model.pkl')// loaded_model = joblib.load('california_housing_model.pkl') **NumPy** np.array([1,2,3],[4,5,6]) # From list // np.zeros((2, 3)) // np.ones((3, 2)) // np.full((2, 2), 7) # 2x2 array filled with 7 // np.arange(0, 10, 2) # Numbers from 0 to 10 with step 2 // np.linspace(0, 1, 5) # 5 evenly spaced numbers from 0 to 1// np.dot(A, B) # matrix multiplication // Shape: (2, 3) arr.shape // Number of dimensions: 2 arr.ndim // Size: 6 arr.size // Data type: int64 arr.dtype // Item size: 8 arr.itemsize // **Indexing and Slicing** Indexing starts from 0, Use slices to extract parts of the array: [start:stop:step] . (inclusive of start, exclusive of stop) **Reshaping Arrays** arr = np.arange(12) // arr.reshape(3, 4) # 3 rows, 4 columns // # Split into 3 equal parts np.split(arr, 3) # Use np.array_split if the array cannot be split evenly // np.round(arr, 2) // np.sort(arr) # ascending // np.sort(arr)[::-1] # descending // np.vstack((arr1, arr2)) np.hstack((arr1, arr2)) // np.mean(arr) // np.median(arr) // np.std(arr) // np.var(arr) // np.exp(arr): This calculates the exponential e^x // np.log(arr): This calculates the natural logarithm // np.sin(arr): Computes the sine of angles in radians // np.cos(arr): Computes the cosine of angles in radians. // Broadcasting automatically "expands" the smaller array to match the larger array's shape. // np.dot(A, B) # matrix mult // A.T # transpose // np.linalg.inv(A) # inverse // np.random.rand(5) # Uniformly distributed between 0 and 1 // np.random.randn(5) # Normally distributed // mask = arr > 25 -> arr[mask] // row_mask = np.all(arr > 25, axis=1) -> selected_rows = arr[row_mask] // col_mask = np.any(arr > 50, axis=0) -> selected_columns = arr[:, col_mask] // mask = (arr >= 30) & (arr <= 70) **Pandas** pd.Series([10, 20, 30, 40]) // pd.Series(data, index=labels) // arr = np.array([100, 200, 300]) -> myseries = pd.Series(arr, index=['x', 'y', 'z']) // pd.Series({'apple': 3, 'banana': 5, 'cherry': 7}) // myseries['banana'] or myseries[1] // **df.iloc[]** # integer location // df.loc[] # label location // df.iloc[:2] # select first two rows // df.iloc[:2, :2] # Select first two rows and first two columns // df.loc['a':'b'] # Select rows by label range // pd.read_csv('data.csv', sep=';') // df['Name'] or df.Name # Accessing a column by name // filtered_df = df[df['Age'] > 30] # Select rows where age is greater than 30 // df.sort_values(by='Age') // df.groupby('City')['Age'].mean() // df.isnull() # Check for missing values // df.fillna(0) # Fill missing values // df.set_index('Product_ID', inplace=True) # Set 'Product_ID' column as the index // df.reset_index(inplace=True) # Reset index // df[(df['Quantity'] > 0) & (df['Price'] > 20)] // AND & OR | NOT ~ //